PROBLEM STATEMENT #44 | Developer Tools & IT Operations

Database Query Performance Profiler

1. Problem Context & Overview

A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests.

Target Stakeholders / Actors: *Database Administrator, Backend Lead*

2. Sample Functional Requirement (FR) Guideline

Sample Requirement: FR-001 [Priority: High]
Description: The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.
Sample Acceptance Criteria: Pass: Missing composite index identified on filtered columns; Fail: Query plan with sequential scan marked optimized.

3. Sample Non-Functional Requirement (NFR) Guideline

Sample Requirement: NFR-001 [Type: Performance & Security]
Description: Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries or slowing host DB.
Sample Acceptance Criteria: Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.

Deliverables to Submit: [All through a GitHub Repo in your account]

- **Complete Requirements Table:**
  - >> Exactly 5 Functional Requirements (FR-001 to FR-005)
  - >> 2 Non-Functional Requirements (NFR-001 & NFR-002) with
    - ID, Type, Description, Priority, Acceptance Criteria, and Rationale.
- **UML Use-Case Diagram:**
  - >> Model all actors, primary use cases, and include at least one «include» and one «extend» relationship.
- **Use-Case Flow Specification:**
  - >> 1-page document for one core use case detailing Preconditions, Postconditions, Main Success Scenario
  - >> One Alternate Flow.